Ideation Phase

Brainstorm & Idea Prioritization Template

Date	25 September 2026
Team ID	SWTID-2026-3161
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field Value
Maximum Marks	4 Marks

Brainstorm & Idea Prioritization Template:

Brainstorming gives a team an open space to think freely and generate as many ideas as possible before judging them. Quantity is favoured over quality at this stage, unusual ideas are welcomed and built upon, and everyone is encouraged to add on to each other's thinking so the team ends up with a wide range of possible solutions.

Use this template to run the brainstorming session for the Script-Controlled ACL project, so the team can explore different ways to solve the field-based access restriction problem before deciding which approach to build.

Reference: https://www.mural.co/templates/brainstorm-and-idea-prioritization

Step-1: Team Gathering, Collaboration and Select the Problem Statement

Define your problem statement (How Might We):

How might we restrict a user's access to a record dynamically, based on the value stored in one of its fields, without creating a separate static role for every possible value?

Key rules of brainstorming, to run a smooth and productive session:

- Stay on topic.
- Defer judgment.
- Go for volume.
- Encourage wild ideas.
- Listen to others.
- If possible, be visual.

Step-2: Brainstorm, Idea Listing and Grouping

Write down any ideas that come to mind that address the problem statement, then cluster similar or related ideas into themed groups.

Group 1 – Scripting the ACL Condition

- Write the field-value check directly inside the ACL's own Script field.
- Move the logic into a reusable Script Include and call it from the ACL condition.
- Use GlideRecord to read the record's field and compare it to the requesting user's attributes.

Group 2 – Alternatives to a Script-Controlled ACL

- Use a Business Rule to hide the field instead of blocking the whole record.
- Use a Data Policy to make the field read-only for certain users.
- Combine a role-based ACL with a condition for the simple cases, and script only the dynamic ones.

Group 3 – Getting the User's Attributes

- Use gs.getUser() / gs.getUserID() to fetch the requesting user's department, role, or clearance level.
- Store the comparison value (e.g., department) on the user record so the script only needs one lookup.

Group 4 – Performance & Security

- Index the field the ACL script filters on, since it will run on every record access.
- Keep the script short and avoid nested queries that could slow down list views.
- Log denied-access attempts for audit purposes.

Group 5 – Testing the ACL

- Create test users with different field values to confirm access is granted/denied correctly.
- Use the Automated Test Framework (ATF) to add regression tests for the ACL.
- Test all four operations — Read, Write, Create, Delete — not just Read.

Step-3: Idea Prioritization

Place the idea groups on the grid below to decide which are important and which are feasible to build first.

	Low Feasibility	**High Feasibility**
High Importance	Group 3 – Getting the user's attributes (needs a data-model change first)	Group 1 – Scripting the ACL condition (core solution)
Low Importance	Group 2 – Alternative approaches (explore only if scripting doesn't fit)	Group 4 & 5 – Performance tuning and ATF testing (do alongside the build)

Importance — of these ideas, which would have the most positive impact if it were done, regardless of difficulty or cost?

Feasibility — which ideas are easier to build than others (cost, time, effort, complexity)?